

DevOps Interview Preparation Guide

Sanket Ajay Chopade | DevOps Engineer | 0–2 Years Experience Range

Structured Answers · Scenario Responses · Cross-Question Handling

This document provides structured, point-wise answers to the complete interview question bank tailored for Sanket's resume. Every answer references real tasks from the 9-month internship at **Hisan Labs** and the **Three-Tier DevSecOps** and **Go Cloud-Native GitOps** projects. Answers are written in first-person to help with verbal delivery in interviews.

1. Resume Deep-Dive Questions

Q1. Explain the complete architecture of the microservices-based e-commerce application.

The application was deployed on **Amazon EKS** and consisted of multiple independently deployable microservices — frontend (React/Node), backend APIs (Node.js), and a database layer (RDS). Traffic entered through an **AWS Application Load Balancer (ALB)**, which routed to an **Ingress Controller** inside the Kubernetes cluster. Each microservice ran as a separate **Kubernetes Deployment** with its own **Service**. Inter-service communication was synchronous via REST/HTTP within the cluster DNS. Persistent data was stored in **Amazon RDS** (MySQL/PostgreSQL), with connection strings supplied via Kubernetes **Secrets**.

Why microservices over a monolith?

- Independent deployability: a bug in the payment service does not require redeploying the entire app.
- Technology flexibility: each service can use the most suitable stack.
- Horizontal scalability: HPA can scale only the high-load services independently.
- Fault isolation: failure in one service does not cascade to others.

What challenges did you face?

- Distributed tracing: correlating logs across services required structured logging and Grafana dashboards.
- Secret management: each service needed its own DB credentials — managed via Kubernetes Secrets.
- Network policies: ensuring services could only communicate with their direct dependencies.

How did you trace requests across services?

We used **Prometheus** and **Grafana** for metrics-based correlation. Each service emitted structured logs with a request-ID header that was forwarded through the chain, enabling log filtering in **CloudWatch** and **Datadog**.

■ Scenario: Payment service is down but others are running. How do you troubleshoot?

- Run: `kubectl get pods -n | grep payment` — check pod status.
- Run: `kubectl describe pod` — look for `CrashLoopBackOff`, `OOMKilled`, image pull errors.
- Run: `kubectl logs --previous` — review last crash logs.
- Check: `kubectl get svc payment-svc` — ensure the Service selector matches pod labels.
- Check: `kubectl get endpoints payment-svc` — if empty, label mismatch exists.
- Check AWS Target Group health in ALB console to verify backend targets.
- Review Grafana dashboard for CPU/memory spikes and DB connection pool exhaustion.
- Check Datadog or CloudWatch alerts for recent anomalies or dependency failures.

Q2. Describe your exact responsibilities in the 4-member DevOps team.

- Owned all 3 Jenkins pipelines: Terraform infra provisioning, frontend deployment, and backend deployment.
- Managed Terraform modules for VPC, EKS, RDS, S3, and Security Groups.
- Maintained Kubernetes manifests: Deployments, Services, Ingress, HPA, ConfigMaps, and Secrets.

- Monitored cluster and application health using Prometheus, Grafana, and CloudWatch.
- Managed AWS IAM users, roles, and policies following the Principle of Least Privilege.
- Maintained Jenkins plugins, credentials, and integrations with GitHub, Docker, and AWS.
- Wrote Bash and Python scripts to automate repetitive operational tasks.

How were deployments approved?

Code changes were submitted via Pull Requests on GitHub. Jenkins pipelines were triggered by webhooks on merge to the main branch. Argo CD handled the final sync to the cluster, requiring the Git manifest to be updated before deployment occurred — providing a natural audit trail.

2. Jenkins Pipeline Deep-Dive

Q3. Explain one Jenkins pipeline end-to-end.

Here is the backend application deployment pipeline used in the Three-Tier DevSecOps project:

- **Checkout:** Clones the GitHub repository using a stored Jenkins credential.
- **SonarQube Analysis:** Runs static code analysis. Pipeline pauses at Quality Gate; fails if gate is not passed.
- **OWASP Dependency Check:** Scans third-party libraries for known CVEs using the OWASP plugin.
- **Build & Test:** Runs Maven/npm build and unit tests.
- **Docker Build:** Builds the Docker image using the project Dockerfile.
- **Trivy Scan:** Scans the Docker image for OS and library vulnerabilities before pushing.
- **Docker Push:** Pushes the image to Amazon ECR using AWS credentials stored in Jenkins.
- **Update Manifest:** Updates the image tag in the Kubernetes manifest YAML in the GitOps repo.
- **Argo CD Sync:** Argo CD detects the manifest change and syncs the cluster to the new desired state.

Why Declarative Pipeline over Scripted?

- Declarative is more readable, enforces structure, and supports built-in error handling.
- `post{}` blocks allow clean success/failure notifications without try-catch boilerplate.
- Easier to review in Pull Requests and onboard new team members.

How were secrets stored?

- AWS credentials: stored as Jenkins 'Secret text' or 'AWS Credentials' type.
- Docker registry credentials: Jenkins Credentials with ID referenced in `withCredentials{}` block.
- SonarQube token: stored as Jenkins Secret and referenced via environment variable.
- Kubernetes kubeconfig: stored as Jenkins Secret File and loaded during deployment stages.

■ Scenario: Build passed but deployment never happened — what do you check?

- Check Jenkins console output for the 'Update Manifest' and 'Docker Push' stages.
- Verify the ECR push succeeded: `aws ecr describe-images --repository-name .`
- Check if the GitOps repo manifest was actually updated with the new image tag.
- Open Argo CD UI → check Application sync status. Look for `OutOfSync` or `Error` state.
- Check Argo CD logs: `kubectl logs -n argocd -l app.kubernetes.io/name=argocd-application-controller`.
- Verify Jenkins credentials for the GitOps repo have write access.

■ Scenario: Jenkins agent goes offline during deployment.

- Check Jenkins UI → Manage Nodes → verify agent status.
- SSH into agent node and check Jenkins agent process: `ps aux | grep jenkins`.
- Review agent logs at: `/var/log/jenkins/jenkins-agent.log` or the agent's work directory.
- Check system resources: `df -h` (disk), `free -m` (memory), uptime (CPU load).
- For EC2 agents: check AWS Console for instance health and system status checks.
- To prevent: use auto-scaling agent pools, set agent connection timeout and retry settings, and monitor agent nodes via CloudWatch.

3. Terraform Real-Time Scenarios

Q4. Explain your Terraform project structure and module organisation.

- Root module contains environment-specific tfvars (dev.tfvars, prod.tfvars) and calls child modules.
- Modules: vpc/, eks/, rds/, ec2/, s3/, iam/ — each self-contained with main.tf, variables.tf, outputs.tf.
- Remote state stored in an S3 bucket with DynamoDB table for state locking.
- Separate state files per environment to prevent blast radius in case of a plan failure.

Why remote state with S3 + DynamoDB?

- S3 provides durable, versioned storage — previous states can be recovered if corruption occurs.
- DynamoDB locking ensures only one terraform apply runs at a time, preventing race conditions.
- Local state is not suitable for team environments — anyone with local copy could diverge.

■ Scenario: EC2 instance deleted manually from AWS Console — what happens on next apply?

- Terraform detects drift during terraform plan — it will show '+ aws_instance.x will be created'.
- This is because the state file still holds the resource but it no longer exists in AWS.
- Running terraform apply will recreate the instance as defined in the configuration.
- To explicitly detect drift without applying: run terraform plan and review the diff.
- Best practice: use AWS Config or CloudTrail alerts to detect and alarm on out-of-band changes.

■ Scenario: Terraform state file becomes corrupted.

- Immediately: do NOT run terraform apply — it will try to recreate all resources.
- Recover from S3 versioning: `aws s3api get-object --version-id --bucket terraform.tfstate`.
- If no backup: use terraform import to manually re-import each existing resource into a fresh state.
- Audit the DynamoDB lock table to ensure no stale lock is held.
- Prevention: enable S3 versioning, set lifecycle policies to retain state history for 30+ days.

■ Scenario: terraform apply failed after creating half the resources.

- Run: terraform show — review current state to see what was created.
- Run: terraform plan — to see remaining resources and what failed.
- Address the root cause (e.g., IAM permission, quota limit, resource name conflict).
- Re-run terraform apply — Terraform is idempotent; already-created resources will not be duplicated.
- If rollback is required: terraform destroy -target= for each partially created resource.

4. AWS EKS Production Scenarios

Q5. Explain EKS architecture including control plane, worker nodes, and VPC layout.

- Control Plane: Managed by AWS — runs API Server, etcd, Scheduler, and Controller Manager in AWS-owned account.
- Worker Nodes: EC2 instances (managed node groups) in customer VPC, joined to the cluster via bootstrap script.
- VPC Layout: Public subnets hold the ALB and NAT Gateway; private subnets host worker nodes and RDS.
- Node Groups: Managed Node Groups used for automatic node lifecycle (patching, replacement).
- Networking: Each pod gets a VPC IP via the VPC CNI plugin — no NAT within the cluster.
- Ingress: AWS Load Balancer Controller creates ALB/NLB resources from Kubernetes Ingress/Service annotations.

■ Scenario: Pods are stuck in Pending state.

- `kubectl describe pod` — check Events section for scheduling failures.
- Insufficient resources: 'Insufficient cpu' or 'Insufficient memory' means no node can fit the pod.
- Node selector / affinity mismatch: pod has nodeSelector that no node satisfies.
- PVC not bound: if pod mounts a PVC, check `kubectl get pvc`.
- Taint/Toleration issue: node has a taint the pod does not tolerate.
- Fix resource shortage: scale node group via AWS Console or `aws eks update-nodegroup-config`.

■ Scenario: Pods running but application is inaccessible.

- `kubectl get pods` — confirm Running and READY (e.g., 1/1).
- `kubectl get svc` — verify Service exists and has a ClusterIP or LoadBalancer EXTERNAL-IP.
- `kubectl get ingress` — check ADDRESS is populated; if empty, ALB is not provisioned.
- `kubectl describe ingress` — check annotations for AWS Load Balancer Controller and any error events.

- Test internal: `kubectl exec -it -- curl http://`
- Check Security Groups: ensure ALB SG allows 80/443 and node SG allows ALB traffic.
- Check Route 53 / DNS: verify the CNAME/A record points to the correct ALB DNS name.

■ **Scenario: EKS nodes are NotReady.**

- `kubectl describe node` — check Conditions and Events.
- SSH into the node (via SSM): check kubelet status: `systemctl status kubelet`.
- Check kubelet logs: `journalctl -u kubelet -n 100`.
- Common causes: disk pressure (`df -h`), memory pressure (`free -m`), or network plugin failure.
- AWS side: check EC2 instance health in AWS Console; verify node IAM role has required policies.
- Recovery: cordon the node, drain it, then terminate and let the ASG replace it.

■ **Scenario: HPA is not scaling.**

- `kubectl describe hpa` — check 'Conditions' for 'AbleToScale' and 'ScalingActive'.
- HPA requires Metrics Server: `kubectl get apiservice v1beta1.metrics.k8s.io` — must be Available.
- If Metrics Server is missing: install via Helm or `kubectl apply -f metrics-server.yaml`.
- Verify resource requests are defined in pod spec — HPA calculates utilization relative to requests.
- Check `minReplicas` / `maxReplicas` configuration is appropriate for the workload.

5. Docker Troubleshooting

■ **Scenario: Container works locally but fails in Kubernetes.**

- Check image tag: ensure the same image:tag used locally is pushed to ECR and referenced in the manifest.
- `kubectl logs` — review application startup errors.
- `kubectl describe pod` — look for `ImagePullBackOff` (ECR auth issue) or `CrashLoopBackOff`.
- Environment variables: local `.env` file may not exist in Kubernetes — verify `ConfigMaps` and `Secrets`.
- File system: if app writes to local disk, ensure a PVC is mounted or the app is stateless.
- Network: container may try to connect to `localhost:5432` (local DB) — in Kubernetes, use the Service DNS name.
- Resource limits: pod may be `OOMKilled` if limits are too low. Check: `kubectl describe pod | grep -i oom`.

■ **Scenario: Docker image size is 2 GB — how do you optimise?**

- Use multi-stage builds: build stage uses full SDK image; final stage copies only compiled artifact.
- Use `Distroless` or `Alpine` base images for the final stage (as done in the Go GitOps project).
- Remove build dependencies and package manager caches within the same RUN layer.
- Use `.dockerignore` to exclude `node_modules`, test files, git history, and documentation.
- Order layers so frequently changing files (source code) are at the end to maximise cache reuse.

Example multi-stage Dockerfile (as used in the Go project):

```
FROM golang:1.21 AS builder
WORKDIR /app
COPY . .
RUN go build -o main .

FROM gcr.io/distroless/base-debian11
COPY --from=builder /app/main /
CMD ["/main"]
```

■ **Scenario: Container exits immediately.**

- `docker run` — observe exit code. Exit 1 = application error, exit 137 = `OOMKilled`.
- `docker logs` — review `stdout/stderr` for startup errors.
- `docker run -it /bin/sh` — open a shell to debug the container filesystem and env vars.
- In Kubernetes: `kubectl logs --previous` to see logs from the last failed container.
- Check `ENTRYPOINT/CMD` in Dockerfile — if the process exits, the container stops.

6. GitHub Actions + Argo CD GitOps Scenarios

Q6. Explain GitOps and how Argo CD works.

- GitOps treats Git as the single source of truth for infrastructure and application state.
- Any change to the cluster is made by committing to Git — no manual `kubectl` apply in production.
- Argo CD runs inside the cluster and continuously polls the Git repository for changes.
- On detecting a diff between Git state and cluster state, Argo CD syncs (applies) the manifests.
- This provides full audit history, easy rollbacks (`git revert`), and drift detection.

How does Argo CD detect cluster state vs Git state?

Argo CD periodically fetches manifests from Git and compares them to live Kubernetes resources via the Kubernetes API. If a resource in the cluster differs from the Git manifest (even if changed manually with `kubectl`), Argo CD marks the application as **OutOfSync**. With auto-sync enabled, it will reconcile automatically. Without it, a manual sync is required.

■ **Scenario: Code merged successfully but deployment never happened.**

- Check GitHub Actions workflow run — verify all steps including 'Update Image Tag' step passed.
- Check if the image tag update commit was pushed to the GitOps repo (verify git log on that repo).
- Open Argo CD UI → check Application status — if Synced and Healthy but old version, image tag may not have changed.
- Verify the Argo CD Application spec points to the correct repo path and branch.
- Check Argo CD application controller logs: `kubectl logs -n argocd deploy/argocd-application-controller`.

■ **Scenario: Developer manually changed deployment.yaml in the cluster.**

- Argo CD marks the application OutOfSync within the next polling interval (default: 3 minutes).
- If auto-sync is enabled: Argo CD automatically reverts the change to match Git state.
- If auto-sync is disabled: Argo CD shows OutOfSync status; manual sync required to reconcile.
- This is the core value of GitOps — accidental or unauthorised changes are automatically detected and can be reversed.
- Best practice: restrict `kubectl` access in production; all changes go through Git.

7. Monitoring & Incident Management

■ **Scenario: CPU suddenly reaches 95%.**

- Check Grafana dashboard: CPU utilization per node and per pod.
- Identify top CPU-consuming pods: `kubectl top pods --sort-by=cpu -A`.
- Determine if it is infrastructure (all nodes high) or application (specific pod/deployment).
- If specific pod: `kubectl logs` — look for tight loops, unresponsive external calls.
- Check if HPA has already triggered: `kubectl get hpa`.
- Temporary relief: manually scale the deployment: `kubectl scale deployment --replicas=5`.
- Long-term: investigate application code, add CPU resource limits, and tune HPA thresholds.

■ **Scenario: Memory usage keeps increasing — potential memory leak.**

- Observe Grafana memory chart over time — steady upward trend confirms a leak.
- Identify the pod: `kubectl top pods --sort-by=memory`.
- Check if the pod eventually gets OOMKilled: `kubectl describe pod | grep -i oom`.
- Review application logs for unclosed DB connections, large in-memory caches, or recursive calls.
- Mitigation: set memory limits so Kubernetes OOMKills and restarts the pod automatically.
- Long-term: fix the leak in application code; add memory leak profiling in staging.

■ **Scenario: Response time increased from 200ms to 3 seconds.**

- Check Grafana/Datadog for the time the spike began — correlate with recent deployments.
- Check database metrics: RDS CloudWatch — slow queries, high DB CPU, connection pool exhaustion.
- Check pod logs for slow external API calls or downstream service timeouts.
- Network: check AWS ALB access logs for request duration and target response time.
- Run: `kubectl top pods` — check if CPU throttling is occurring (container hitting CPU limit).
- Use Datadog APM traces or structured logs with request IDs to isolate the slow component.

8. Security & DevSecOps Questions

Q7. What does SonarQube analyse and what is a Quality Gate?

- SonarQube performs static analysis on source code for: Bugs, Vulnerabilities, Code Smells, Duplications, and Test Coverage.
- Quality Gate is a pass/fail threshold set by the team (e.g., coverage > 80%, no critical vulnerabilities).
- In our Jenkins pipeline, the stage pauses after analysis and fails the build if the gate is not passed.
- This prevents code with known security issues or insufficient test coverage from being deployed.

SonarQube server unavailable during pipeline?

The Jenkins stage times out and the build fails. We handled this with a try-catch in the Declarative Pipeline's `post{}` block — sending a Slack/email notification with the failure reason. We also set up SonarQube as a high-availability service to minimise downtime.

Q8. Trivy found Critical vulnerabilities — would you continue the deployment?

- No. In our pipeline, Trivy is configured to fail the build on CRITICAL severity findings.
- The team is notified via the Jenkins post-failure notification.
- Developer must update the dependency or base image to a patched version.
- HIGH severity vulnerabilities are reported but deployment may proceed with team approval.
- This policy was defined in the pipeline threshold flags: `--exit-code 1 --severity CRITICAL`.

Q9. Developer requests AdministratorAccess on AWS — how do you respond?

- Deny the request. AdministratorAccess violates the Principle of Least Privilege.
- Ask: what specific actions does the developer need? (e.g., S3 read, ECR push, EKS describe).
- Create a custom IAM policy granting only the required actions on the required resources.
- Attach the policy to a role, and allow the developer to assume the role — not a permanent attachment.
- For production access: require MFA and time-bound role assumptions via AWS STS.
- Periodically review and audit IAM permissions using AWS IAM Access Analyzer.

9. Linux Production Scenarios

■ Scenario: Root filesystem at 100%.

- `df -h` — confirm which filesystem is full.
- `du -sh /*` — identify the largest top-level directories.
- `du -sh /var/log/*` — logs are the most common culprit.
- `find / -size +500M -type f 2>/dev/null` — locate large files.
- `lsof | grep deleted` — find deleted files still held open by processes (won't free space until process restarts).
- Immediate relief: truncate or rotate large log files, remove old Docker images (docker system prune).
- Long-term: configure logrotate, set Docker log driver with size limits, monitor disk via CloudWatch.

■ Scenario: Server load suddenly becomes 20.

- `uptime` — confirm load average values (1m, 5m, 15m).
- `top` or `htop` — identify processes consuming CPU.
- `iostat -x 1` — check I/O wait (wa). High I/O wait = disk bottleneck, not just CPU.
- `vmstat 1` — check for high si/so (swap in/out = memory pressure).
- `ps aux --sort=-%cpu | head -20` — identify runaway processes.
- If a single process: `kill -15` to gracefully stop, then investigate.
- If I/O related: check RDS slow query logs or application DB connection patterns.

■ Scenario: Application port not accessible.

- `ss -tulpn | grep` — verify the process is listening on the expected port.
- If not listening: restart the application service; check application logs.
- `curl -v http://localhost:` — test connectivity from the server itself.
- If local works but remote doesn't: check firewall — `ufw status` or `iptables -L`.

- In AWS: check Security Group inbound rules — ensure the port is open from the correct source.
- telnet — test TCP connectivity from the client side.
- traceroute — identify network routing issues.

10. Behavioural & Ownership Questions

Q10. Tell me about a production issue you solved.

Situation: During my internship at Hisan Labs, Prometheus alerted on high memory usage for one of the backend microservice pods, which eventually triggered OOMKilled restarts during peak hours.

Action: I checked `kubectl top pods` and confirmed the affected pod. I reviewed the application logs and found that a background job was loading entire database result sets into memory without pagination. I escalated to the development team with the log evidence. Meanwhile, I increased the memory limit in the Kubernetes manifest as a temporary relief measure, applied it via the GitOps workflow, and updated the Grafana alert threshold.

Result: The restarts stopped within 30 minutes. The development team fixed the pagination bug in the next sprint. I added a memory usage alert at 70% threshold to catch this earlier in future.

Q11. Tell me about a deployment failure.

Situation: A Jenkins pipeline pushed a new backend Docker image to ECR, but Argo CD sync failed silently — the old version kept running.

Root Cause: The GitHub Actions step that updated the image tag in the GitOps manifest used an expired Personal Access Token. The git push succeeded locally in testing but silently failed in CI.

Resolution: Rotated the PAT and re-ran the pipeline. To prevent recurrence, I added an explicit git push exit-code check in the workflow and set up a PAT expiry reminder in GitHub.

Q12. Multiple alerts triggered simultaneously — how do you prioritise?

- First: assess customer impact — is the application returning errors to users? That takes priority.
- Acknowledge all alerts in Datadog/PagerDuty to signal active investigation.
- Triage by severity: production down > degraded performance > non-critical background job.
- Notify the team lead and relevant stakeholders with a brief impact statement within 5 minutes.
- Focus on one issue at a time — assign teammates to parallel investigate lower-priority alerts.
- Post a status update every 15 minutes until resolved.
- After resolution: write a brief postmortem covering root cause, timeline, and preventive measures.

11. Most Difficult Cross-Questions — Senior Interviewer Level

Q13. Explain packet flow from browser to pod in EKS.

- User types URL → DNS resolves to ALB IP (via Route 53 CNAME).
- Browser sends HTTP/HTTPS request to ALB on port 80/443.
- ALB listener rule matches and forwards to Target Group (worker node IP + NodePort or pod IP via IP mode).
- With ALB in IP mode (AWS Load Balancer Controller): request goes directly to pod IP in the VPC.
- Kubernetes kube-proxy (or eBPF on newer setups) routes traffic to the correct pod.
- Pod receives request on the container port defined in the Deployment spec.
- Response follows the reverse path back to the browser.

Q14. What happens internally when you run terraform apply?

- Terraform reads all `.tf` files and builds an in-memory dependency graph of resources.
- It loads the remote state from S3 and acquires the DynamoDB lock.
- `terraform plan` is implicitly run — Terraform calls AWS APIs to compare current state with desired state.
- Terraform presents the diff (create/update/destroy). On confirmation, it executes in dependency order.
- Each resource change calls the appropriate AWS SDK/API (e.g., `CreateVpc`, `CreateSubnet`).
- After each successful resource operation, Terraform writes the new state to S3.
- On completion, the DynamoDB lock is released.

Q15. Explain how Argo CD knows cluster state differs from Git state.

- Argo CD Application Controller continuously polls Git (default every 3 minutes) for manifest changes.
- It also watches Kubernetes API for live resource changes using informers.
- It computes a desired state hash from Git manifests and a live state hash from the cluster.
- If hashes differ → Application is marked OutOfSync.
- The diff is computed field-by-field — ignoring annotations/labels that Kubernetes adds automatically (configurable via ignoreDifferences).
- With auto-sync: controller triggers a sync (equivalent to kubectl apply) to reconcile.

Q16. Walk through your Grafana alert incident response process.

- Alert fires → PagerDuty/Slack notification received → acknowledge within SLA (e.g., 5 min).
- Open the alerting Grafana panel — check the metric, time of spike, and affected service/node.
- Cross-reference with recent deployments: was anything deployed in the last 1 hour?
- Check correlated metrics: high CPU often accompanies high memory or I/O — look at adjacent panels.
- kubectl top pods and kubectl logs for application-level context.
- If application issue: notify development team. If infrastructure: take remediation action.
- Communicate status to stakeholders every 15 minutes.
- Resolve → close alert → write postmortem documenting timeline, root cause, and prevention steps.

Prepared specifically for Sanket Ajay Chopade based on resume from Hisan Labs internship and project experience. Answers reflect hands-on tasks performed and are ready for verbal delivery in interviews.